Ask-about-Aditya — Glass-Box RAG Chatbot

Personal project · September 2026 · adityamukhopadhyay.vercel.app/ask · github.com/adityamukhopadhyay/portfolio-rag (private)

A retrieval-augmented chatbot embedded in my portfolio that answers questions about my work, in the first person, from my own project documents — and shows its entire retrieval pipeline for every question. This document is written so that the whole system can be grasped at a glance and defended in an interview: what it is, how it is built, what was measured, what broke, and why each decision was made.

At a glance

- **What it is:** a RAG assistant on my portfolio. Visitors ask "do you know X?" / "how did you build Y?"; it answers as me, cites the document, and streams every pipeline step live to an inspector panel.
- **Corpus:** 38 markdown documents (~20k words, 290 sections) generated from 20 project .docx files, my resume, an engineering story bank, the site's own pages, a hand-written facts file with negative knowledge, and this project's own document — all redacted before use.
- **Retrieval:** hybrid dense (Gemini embedding, 768-d) + sparse (BM25 written from scratch, IDF inside Qdrant), fused with reciprocal-rank fusion computed client-side so every list is inspectable; optional query expansion and listwise LLM reranking.
- **Two pipelines A/B-tested:** A = heading-aware sections; B = LLM-extracted atomic propositions that retrieve their parent section. Same answerer, same evaluation.
- **Result:** every configuration put the ground-truth section in the final five 98.9% of the time. The reranker and expansion added ~1.1 s each for no measurable gain, so production runs A with both off (2.3 s median) — they remain one click away and fully traced.
- **Hosting:** FastAPI + Qdrant on Railway (private network, persistent volume), provisioned through Railway's GraphQL API; Gemini for embeddings, rerank and generation; Next.js front end on Vercel. Nothing local in production.
- **Cost control:** answer cache with pinned pre-drafted questions, 5 live questions per visitor per day (cached answers free), per-IP and global limits, thinking switched to minimal on every model call.
- **Let's connect:** a booking form (day chips, time slots in the visitor's zone, optional note) emails me instantly and hands the visitor a Google Calendar link and an .ics with both of us as attendees — the invite lands on my calendar from theirs.
- **Live results:** stress 18/18; load 120/120 at 20 concurrent, p50 1.96 s, p95 2.81 s, 8 req/s on one instance.

Number	Value	Where it comes from
Documents / sections indexed	38 / 290	/health, live
Propositions (pipeline B)	≈2,400	ingest log
Embedding	gemini-embedding-001, 768-d	config
Dense / sparse top-K, fused candidates, final passages	30 / 30 → 20 → 5	config
Benchmark	30 curated + 60 synthetic + 16 adversarial	eval/
hit@final (all configs)	98.9%	EVAL_REPORT.md
Median latency, winner vs with rerank	2.3 s vs 3.6 s	EVAL_REPORT.md
Adversarial pass	15 / 16	EVAL_REPORT.md
Per-visitor quota (prod)	5 live questions / 24 h (+ IP cap 20)	RATE_LIMIT_PER_USER
Live load test	120/120 OK · 20 concurrent · p50 1.96 s · p95 2.81 s	eval/results/load_*.json

1. Summary

Ask-about-Aditya is a retrieval-augmented generation (RAG) chatbot embedded in my portfolio website. It answers questions about my work, skills and projects from my own project documents, cites the document each claim came from, and speaks in the first person as an AI version of me: a visitor's "you" means Aditya. Its distinguishing feature is that it is a glass box — for every question, the interface streams the full retrieval pipeline as it runs: the persona rewrite, the expanded queries, the dense and sparse top-K with scores, the reciprocal-rank fusion, the reranker's before-and-after ordering with a one-line reason per passage, the prompt that was built, and every model call with its latency and token counts.

I built it in a single day in September 2026, from corpus conversion through deployment. Two retrieval pipelines were built and evaluated against each other on a judged benchmark; the winner is persisted as the production configuration and the finding — that the heavier stages did not earn their latency on this corpus — is stated openly on the portfolio page.

2. Tech Stack & Infrastructure

- **Backend:** Python 3.11, FastAPI, Server-Sent Events (sse-starlette), pydantic-settings
- **Vector database:** Qdrant (open source) on Railway with a persistent volume, private networking; embedded local mode for development and evaluation
- **Embeddings:** Gemini gemini-embedding-001, 768 dimensions (Matryoshka truncation), unit-normalised, dot-product similarity
- **Sparse retrieval:** BM25-style encoder written from scratch: hashed term ids, tf saturation; IDF applied inside Qdrant's sparse index
- **Models:** gemini-3.5-flash (answers, evaluation judge); gemini-3.5-flash-lite (query expansion, listwise rerank, proposition extraction)
- **Front end:** Next.js 16 static export on Vercel, React 19, Tailwind v4; floating widget on every page plus a full /ask page with the inspector
- **Infrastructure:** Railway project provisioned via its GraphQL API (services, volume, variables, domain); GitHub private repo; Vercel
- **Corpus tooling:** python-docx for the 20 project documents; Node's native TypeScript stripping to read the site's content files; pattern-based redaction; hand-written facts file

3. System Architecture

Three runtime parts and one build-time part. The browser talks only to the API over SSE; the API talks to Qdrant over Railway's private network and to Gemini over HTTPS; the corpus is generated offline and indexed on the server.

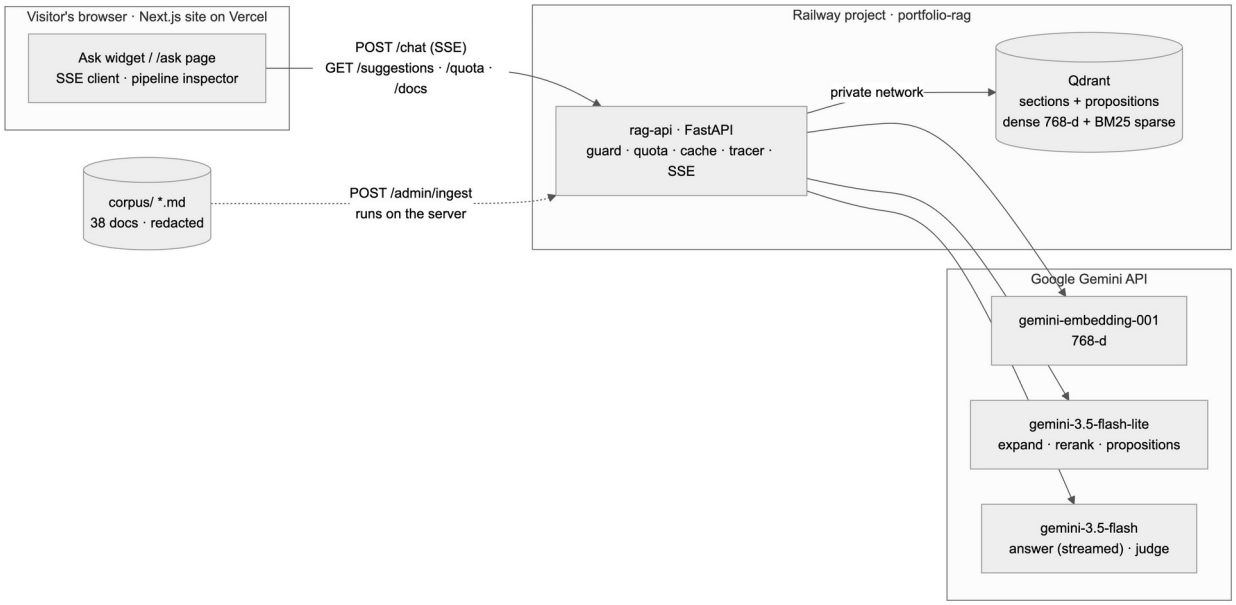


Figure 1 — System overview: browser ↔ rag-api ↔ Qdrant, with Gemini for embeddings, light-model steps and generation.

3.1 Corpus pipeline (build time)

A converter turns every source about me into a canonical markdown corpus with frontmatter (title, slug, kind, page URL): the 20 project .docx files with headings, lists, tables and embedded screenshots preserved; the resume source of truth; the engineering story bank; the site's own profile and project pages; a hand-written facts file; and this project's own document. Every string passes through a redaction layer — emails, phone numbers, secrets, internal hostnames, IP addresses, a denylist — because the corpus is served to the public internet. Each document carries its portfolio page URL so citations link to the human-readable page rather than the raw file.

Negative knowledge. The facts file states what I do not use (LangChain, LlamaIndex, CrewAI) and what is not on record, so "do you use LangChain?" is answered from a stated fact rather than from the absence of evidence.

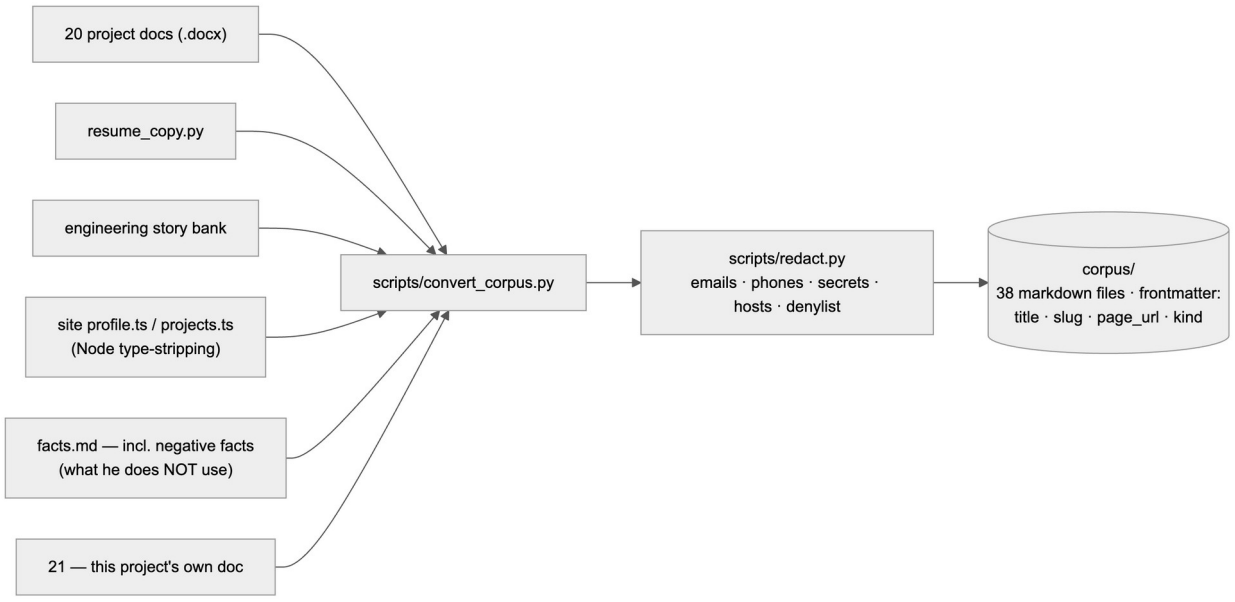


Figure 2 — Corpus pipeline: six source types → converter → redaction → 38 markdown documents.

3.2 Indexing — the A/B variable

Both pipelines split documents into heading-aware sections (tiny sections merged into their neighbour, long ones split at ~380 words on paragraph boundaries) and store each unit with a dense vector and a sparse BM25 vector in Qdrant. What differs is the unit that is embedded and searched.

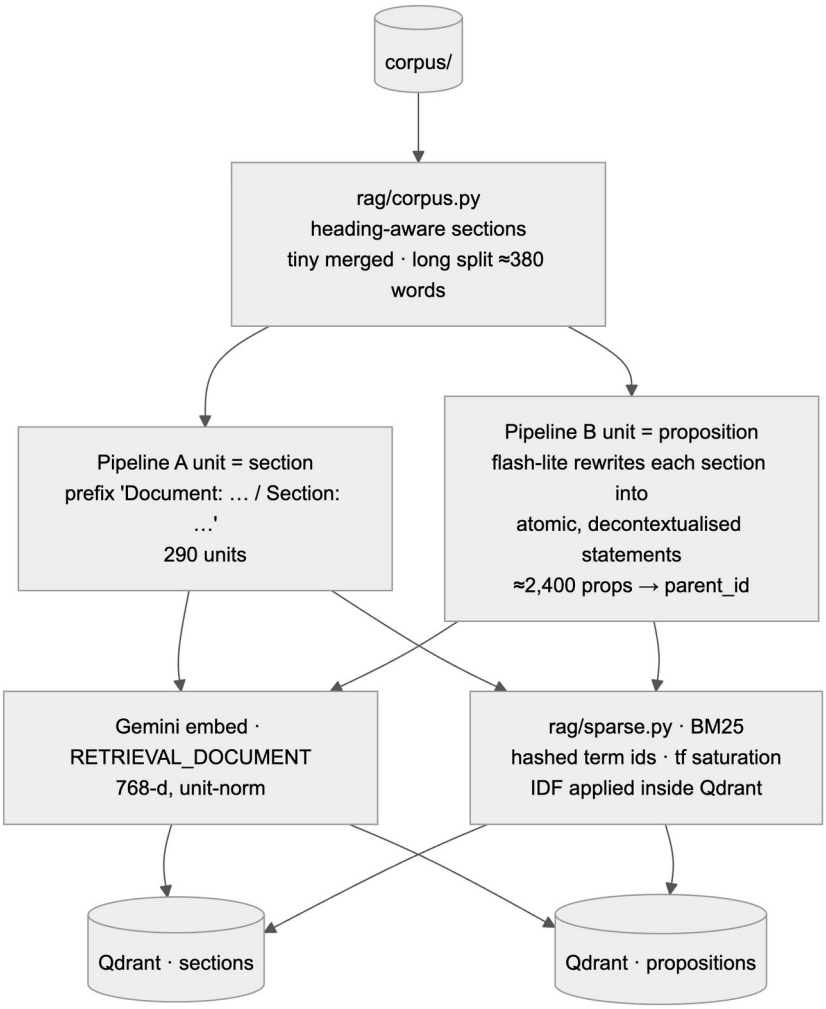


Figure 3 — Indexing: section units (A) and proposition units (B) share the same embedding and sparse encoders.

	Pipeline A — sections	**Pipeline B — propositions → parent**
Unit indexed	One markdown section, embedded with a deterministic prefix "Document: … / Section: …" so the vector carries context	Atomic, decontextualised statements extracted from each section by flash-lite (subject always named)
What the answerer reads	The section	The parent section of the matched propositions (small-to-big)
Cost at ingest	290 embeddings	290 extraction calls + ≈2,400 embeddings
Intuition	Cheap, robust, keeps context together	Precise matching on facts, immune to pronoun/context loss

3.3 Query pipeline (per request)

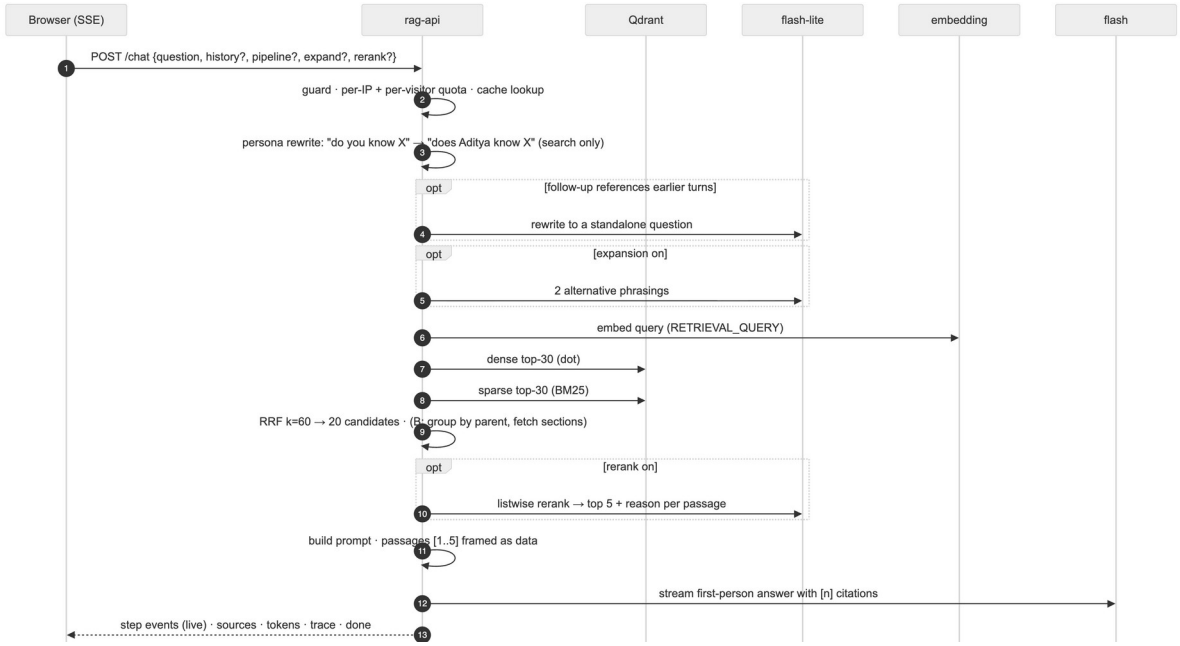


Figure 4 — One request end to end; every numbered step is a traced stage streamed to the inspector.

1. **Guard.** NFKC normalisation, control characters stripped, 1,000-character cap, injection and PII pattern flags (informational — the model is forewarned, not blocked).
2. **Limits and cache.** 20 requests/min per IP, 400/day global, 5 live questions per visitor per 24 h; the answer cache is keyed on the normalised question + pipeline. A hit replays the answer, sources and original trace in single-digit milliseconds and costs no quota.
3. **Persona rewrite.** Visitors say "you"; the documents say "Aditya". A deterministic regex step rewrites second person to third for the search query only ("Do you know MCP?" → "does Aditya know MCP?"); the answerer still sees the original wording.
4. **Follow-up rewrite (optional).** If the question references earlier turns ("that one", "it"), flash-lite rewrites it to stand alone.
5. **Query expansion (optional, off by default).** flash-lite writes two alternative phrasings; each is searched and the per-query lists are fused again.
6. **Embed.** gemini-embedding-001 with task type RETRIEVAL_QUERY, 768-d, unit-normalised (~550–700 ms).
7. **Dense + sparse search.** Top-30 by dot product and top-30 by BM25 from Qdrant (a few milliseconds each on this corpus).
8. **Fusion.** Reciprocal-rank fusion, k = 60, computed on the client so the dense list, the sparse list and the fused list are all visible; 20 candidates kept. Pipeline B additionally groups proposition hits by parent (a section scores the sum of its top-3 propositions) and fetches the parent text.
9. **Rerank (optional, off by default).** One listwise call to flash-lite with the 20 candidates returns an order plus a one-line reason per passage (~1.1 s).

10. **Prompt.** The final five passages are numbered [1]–[5] and framed as data; the system prompt requires first-person voice, citations, and a plain "I haven't written that down" when the passages do not cover the question.
11. **Generate.** gemini-3.5-flash streams the answer (~2 s); output is scrubbed for non-public contact details and cached with its trace.

3.4 Serving, guards and cost control

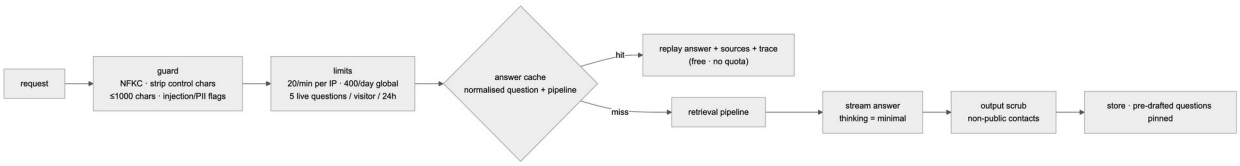


Figure 5 — Request path with the guard, limits and cache in front of the pipeline.

Prompt-injection hardening: passages are labelled as data and the system prompt instructs the model to ignore embedded instructions; the adversarial set exercises this. **Thinking off:** Gemini 3.x thinks by default and the thoughts count against max_output_tokens — with defaults a 700-token budget produced 26 answer tokens. Every generation call runs with thinking_level = minimal (thinking_budget = 0 is rejected by flash-lite in JSON mode). **Pre-drafted questions:** twenty second-person questions are answered once at startup and pinned in the cache with their full trace, so tapping one is instant, free, and still shows how the answer was produced.

4. Key Engineering Features

- **Glass-box tracing.** A per-request Tracer is bound to a context variable; the Gemini client records every embed, generate and stream call against it with no plumbing through the retrieval code. Steps carry their real start time, so the inspector's timeline adds up exactly.
- **Live step streaming.** The API runs retrieval in a worker thread and forwards each finished step to the SSE stream through an asyncio queue, so the browser shows "Embed 552 ms → Dense search 2 ms → …" while the answer is still being generated.
- **Client-side RRF.** Qdrant can fuse server-side, but then only the fused list is visible. Fusing in the API costs nothing on this corpus and makes dense-vs-sparse behaviour inspectable ("in both lists: 17, dense only: 2, sparse only: 1").
- **BM25 without a model.** Terms are hashed to stable ids with tf saturation on the client and IDF applied inside Qdrant's sparse index, so the collection is self-contained and exact technical tokens (pgvector, HDBSCAN, AiSensy) match reliably.
- **First-person persona with retrieval-safe rewriting.** The answer is in Aditya's voice; retrieval is done on a third-person rewrite so the corpus vocabulary matches; the rewrite is shown as its own inspector row.

- **Quota that cannot be gamed by cached traffic.** Only questions that reach the models count; cached and pre-drafted answers are free; the composer shows the remaining count and disables at zero.
- **Provisioning through Railway's GraphQL API.** The CLI rejected a workspace-scoped token, so services, the Qdrant volume, variables and the domain are created idempotently from a script that forwards secrets straight from the environment file without printing them. Pipeline settings are pushed from the evaluation's WINNER.json.
- **Ingestion on the server.** POST /admin/ingest (bearer-protected) builds both indexes where they are served; the local index is only for development and evaluation.

5. UI / UX

The chat is deliberately minimal and framed as a conversation with me: my portrait with a status dot, "AI version of me · answers from my 38 project documents" (counts read live from /health), a first-person greeting, four suggested questions with a "more" link, and a composer with the send button inside the field. Answers are plain text with superscript citation chips that show the source on hover, and an icon row — sources, inspect, copy — each with a tooltip.

The inspector is a separate panel (the right column on /ask, a second column in the widget): a summary line, a thin timeline of the steps that took real time, then one row per step with a line icon and its duration, every row collapsed until opened. The fusion row lists candidates with their dense and sparse ranks; the rerank row shows each passage's move ("15 → 2 ▲13") with the model's reason; model calls fold away at the bottom. Retrieval settings (pipeline A/B, expansion, rerank) sit behind a single settings glyph as segmented controls; "auto" means the server's default, which is the A/B winner.

6. Evaluation

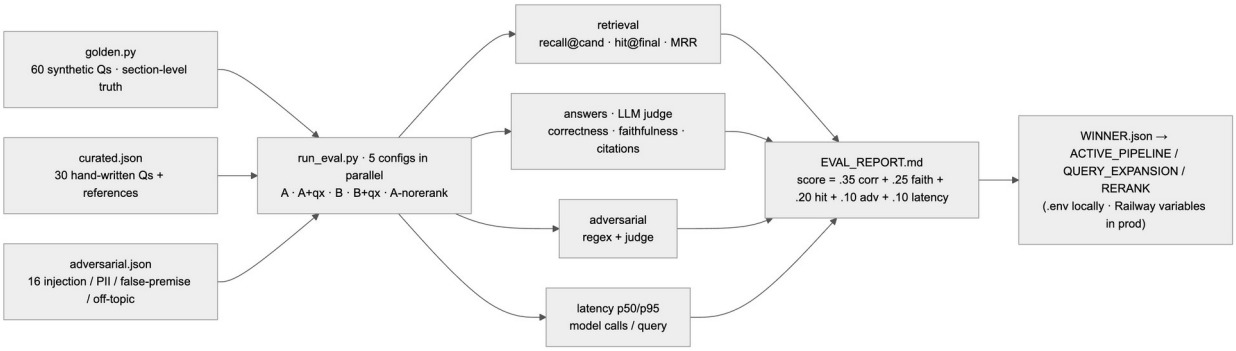


Figure 6 — Evaluation harness: three question sets, five configurations, four metric families, one persisted winner.

Question sets: 30 hand-written questions with reference answers and expected documents; 60 synthetic questions generated per section by flash-lite with section-level ground truth; 16 adversarial cases (instruction override, PII fishing, false premises, off-topic, Hinglish, emoji-only, oversized input). Five configurations — A, A + expansion, B, B + expansion, A without rerank — each ran in its

own process on an identical copy of the index. Retrieval was scored by recall at the candidate stage, hit rate in the final five and MRR; answers were judged by gemini-3.5-flash for correctness against the reference, faithfulness to the passages and citation validity; adversarial cases combined regex checks with a judge. Composite score = 0.35 correctness + 0.25 faithfulness + 0.20 hit@final + 0.10 adversarial + 0.10 normalised latency.

Config	Score	Correct	Faithful	Citations	hit@final	Recall@cand	MRR	Adversarial	p50	p95	Calls/q
A · no rerank	0.987	0.987	1.000	0.992	0.989	1.000	0.866	0.938	2.29 s	3.15 s	2
A	0.932	0.982	1.000	0.992	0.989	1.000	0.869	0.938	3.59 s	4.57 s	3
B	0.931	0.987	0.992	0.988	0.989	0.989	0.862	0.938	3.62 s	4.33 s	3
A + expansion	0.892	1.000	1.000	0.992	0.989	1.000	0.858	0.938	4.72 s	5.86 s	4
B + expansion	0.889	0.993	0.998	1.000	0.989	1.000	0.872	0.938	4.72 s	5.76 s	4

What the numbers say.

- Retrieval is not the bottleneck: every configuration puts the ground-truth section in the five passages handed to the answerer 98.9% of the time. With 290 short, self-contained sections and questions that name concrete mechanisms, hybrid dense + BM25 with RRF is already sufficient.
- The reranker did not earn its latency: hit@final and MRR are unchanged with it on or off, and it costs ~1.1 s and one extra model call per question. It stays available and fully traced, but off by default.
- Query expansion likewise: +1.1 s for the same hit rate. Available, off by default.
- Propositions (B) matched sections (A) but did not beat them, at the cost of 290 extraction calls at ingest. A is the default.
- Faithfulness ≈ 1.0 everywhere: the "passages are data; cite or decline" rule holds. Adversarial 15/16 on every config; the consistent miss is an emoji-only input answered cheerfully instead of asking for clarification.

The persisted production settings are ACTIVE_PIPELINE = a, QUERY_EXPANSION = false, RERANK = false. The benchmark was run on the 36-document corpus before this project's own documents were added.

6.1 Stress and load tests on the live deployment

Edge-input stress (eval/stress.py) against the Railway URL: 18 of 18 cases passed — empty and whitespace input 400, over-long input 413/422, control characters, JSON quotes, emoji and non-Latin scripts, HTML, SQL and prompt-injection payloads all answered normally with nothing echoed back,

malformed bodies 422, and an anonymous burst of 30 requests produced 3 × 200 and 27 × 429 from the per-minute limiter with no 5xx.

Concurrency load (eval/load_test.py): 20 concurrent clients, 120 requests, half over SSE, with the admin bearer so the pipeline rather than the limiter is measured — 120 of 120 returned 200 with no errors at 8.1 requests per second on a single Railway instance; latency p50 1.96 s, p90 2.54 s, p95 2.81 s, max 6.0 s; SSE first token p50 1.40 s, p95 2.23 s. Under 20-way concurrency the per-request latency stayed within about 15% of the single-request benchmark, so the service is I/O-bound on Gemini and Qdrant rather than CPU-bound. The same run without the bearer returns 100% 429 — the intended behaviour for a scraper.

End to end on the live site: a cached suggestion answered in 2.1 s with its trace replayed, a live question ran the full traced path with the inspector updating step by step, and a call booked through the modal (Mon 21 Sep 2026, 11:00 IST) reached the inbox within seconds.

7. Failures Found and Fixed

- **Thinking tokens ate the answer budget.** Answers stopped mid-sentence. usage_metadata showed 670 thought tokens and 26 candidate tokens against a 700-token budget. Fix: thinking_level = minimal on every generation call; thinking_budget = 0 is rejected by flash-lite in JSON mode.
- **Pipeline B indexed nothing.** Two prompts contained literal JSON braces that Python's .format() consumed; extraction and expansion failed silently and fell back. Caught by an end-to-end check that B returned zero hits; fixed by escaping.
- **The browser never saw an SSE event.** sse-starlette emits CRLF line endings; the parser split on LF only, buffered the whole response and yielded nothing (Chrome reported the fetch as aborted). Fixed by normalising line endings and flushing the trailing buffer.
- **Timeline bars overflowed.** Steps timed outside the tracer were recorded with their end time as their start. Fixed by passing the real start time; the timeline now sums exactly to the total.
- **Model withdrawn mid-build.** gemini-2.5-flash-lite stopped being served to new API users; the model list was read from the API and the pipeline moved to the 3.5 generation.
- **Embedded Qdrant is single-process.** Running the API and the evaluation on the same local index deadlocked on the storage lock; each process now gets its own index copy, and five evaluation configs ran in parallel that way (3 hours → 40 minutes).

8. Deployment & Operations

- **Railway project portfolio-rag:** service qdrant (image qdrant/qdrant, volume at /qdrant/storage, API key) and service rag-api (Dockerfile, public domain, health check on /health), connected over qdrant.railway.internal:6333.

- **Variables on rag-api:** GEMINI_API_KEY, QDRANT_URL, QDRANT_API_KEY, ADMIN_TOKEN, ACTIVE_PIPELINE / QUERY_EXPANSION / RERANK (from WINNER.json), PUBLIC_DOCS, CORS_ORIGINS, RATE_LIMIT_PER_MIN / PER_DAY / PER_USER.
- **Index build:** POST /admin/ingest with the admin bearer; idempotent (point ids derive from stable section ids); proposition extraction is cached by section hash so re-ingest only touches changed sections.
- **Observability:** /health (collections, models, cache hits, Gemini call count), /admin/stats (limiter and token counters), and the per-request trace itself.
- **Cost envelope:** Qdrant idles at ~100–150 MB, the API at ~250 MB — inside Railway's $5/month tier; Gemini spend is bounded by the quota and the cache.

9. Interview Q&A

Q. Why hybrid retrieval instead of just embeddings?

The corpus is full of rare technical tokens — pgvector, HDBSCAN, AiSensy, n8n — where lexical matching is more reliable than dense similarity, while dense handles paraphrase ("agents that can't lie" → honesty gates). RRF combines the two rank lists without score normalisation. The fusion row in the inspector shows the split: typically 17 of 20 candidates appear in both lists, 2 only dense, 1 only sparse.

Q. Why reciprocal-rank fusion and not weighted score blending?

Dense dot products and BM25 scores live on different scales; blending needs per-corpus tuning. RRF uses ranks only (1/(k+rank), k = 60), is robust, and needs no tuning. I compute it client-side so all three lists remain visible.

Q. What is the difference between pipeline A and B, and which won?

A embeds heading-aware sections with a situating prefix. B extracts atomic, decontextualised propositions with a light model, retrieves those, and returns the parent section — small-to-big retrieval. On a judged benchmark they tied on hit rate (98.9%); B costs 290 extraction calls at ingest, so A is the default.

Q. Your reranker is off by default — isn't that the interesting part?

It is the interesting finding. Listwise reranking changed neither hit@final nor MRR and cost 1.1 s per question, because retrieval already put the right section in the top five almost every time. I kept it available and fully traced, set it off, and wrote that on the portfolio page. Measuring whether a stage earns its latency is the point.

Q. How do you prevent hallucination?

Passages are numbered and framed as data; the system prompt requires a citation for every claim and a plain "I haven't written that down" when coverage is missing; negative facts are stated explicitly in the corpus; the judge measured faithfulness at ≈1.0 across configs. And the citations link to the actual page, so a reader can check.

Q. How does the persona work without breaking retrieval?

Visitors say "you"; documents say "Aditya". A deterministic regex rewrites second to third person for the search query only — "do you know MCP" → "does Aditya know MCP" — and the answerer sees the original. It is shown as its own inspector row.

Q. How is the trace collected without cluttering the retrieval code?

A Tracer object is bound to a context variable per request. The Gemini client looks up the current tracer and records each call; pipeline stages wrap themselves in tracer.step(). Steps are pushed to an asyncio queue and streamed as SSE events while the worker thread is still running.

Q. What is the chunking strategy and why?

Structure-aware: split on markdown headings, merge sections under ~25 words into their neighbour, split sections over ~380 words on paragraph boundaries, no overlap (overlap is a crutch for fixed-size windows). The heading path is prefixed to the text before embedding so the vector carries its context.

Q. How did you build the evaluation set?

Thirty hand-written questions with references and expected documents, sixty synthetic questions generated per section with section-level ground truth so both pipelines are scored at the same granularity, and sixteen adversarial cases with regex must/must-not rules plus a judge.

Q. What did the thinking-token bug teach you?

Read usage metadata before blaming the prompt. Gemini 3.x thinks by default and the thoughts count against max_output_tokens — 670 thought tokens explained a 26-token answer. Also that per-model capability differs: budget 0 fails on flash-lite in JSON mode, minimal works on both.

Q. How do you keep API costs bounded on a public site?

A per-visitor quota of five live questions a day keyed on a client id, cached and pre-drafted answers free and unlimited, per-IP per-minute and global daily limits, thinking off, and the cheapest model for every non-answer step.

Q. Why Qdrant?

Open source, runs as a single container with a volume, native sparse vectors with server-side IDF, payload filtering, and an embedded local mode that made evaluation possible without a server. The same client code targets both.

Q. What would you change at 100× the corpus?

Move RRF back server-side, add payload filtering by document kind, turn the reranker back on (it should start earning its latency once retrieval gets noisier), consider a cross-encoder, and re-run the same benchmark to decide.

Q. How is it deployed and why through the GraphQL API?

Railway hosts the API and Qdrant over a private network. The CLI rejected the workspace-scoped token I had, so I provisioned services, the volume, variables and the domain through Railway's GraphQL API in an idempotent script that reads secrets from the environment file and never prints them.

Q. What does the inspector actually show?

For every question: the persona rewrite, expanded queries if any, embed time and model, dense top-K and sparse top-K with scores, the fused list with each candidate's dense and sparse rank, the reranker's in→out moves with reasons, the prompt's passages and size, and generation timing including first token — plus every model call with latency and tokens.

10. Numbers to remember

Fact	Value
Corpus	38 docs, ~20k words, 290 sections (36 / 267 at evaluation time)
Embedding	gemini-embedding-001, 768-d (Matryoshka), unit-norm, dot product
Sparse	BM25 from scratch; IDF in Qdrant; k1 = 1.2, b = 0.75
Search sizes	dense 30, sparse 30 → RRF (k = 60) → 20 candidates → 5 final
Pipeline B	≈2,400 propositions → 290 parents; section score = Σ top-3 propositions
Models	3.5-flash answers/judge; 3.5-flash-lite expand/rerank/propositions; thinking = minimal
Benchmark	30 curated + 60 synthetic + 16 adversarial; 5 configs; judged
Winner	A, no rerank, no expansion — score 0.987, p50 2.3 s, hit@final 98.9%
Latency cost of rerank / expansion	+1.1 s each, no accuracy change
Limits	20/min per IP, 400/day global, 5 live questions per visitor per 24 h
Answer budget	1,200 output tokens; ~180-word target

11. Media

Figures 1–6 are rendered from the Mermaid sources in docs/ARCHITECTURE.md of the repository (docs/diagrams/01-overview.png … 06-eval.png). The live system is at adityamukhopadhyay.vercel.app/ask; the evaluation report is eval/EVAL_REPORT.md.